

# Docker Deployment Guide

## Docker Deployment Guide — Invitation Management Application

Complete guide for deploying the Invitation Management Application using Docker.

### Table of Contents

- Prerequisites
- Architecture Overview
- Quick Start
- Configuration
- Building & Running
- Common Operations
- Troubleshooting
- Production Deployment
- Maintenance

### Prerequisites

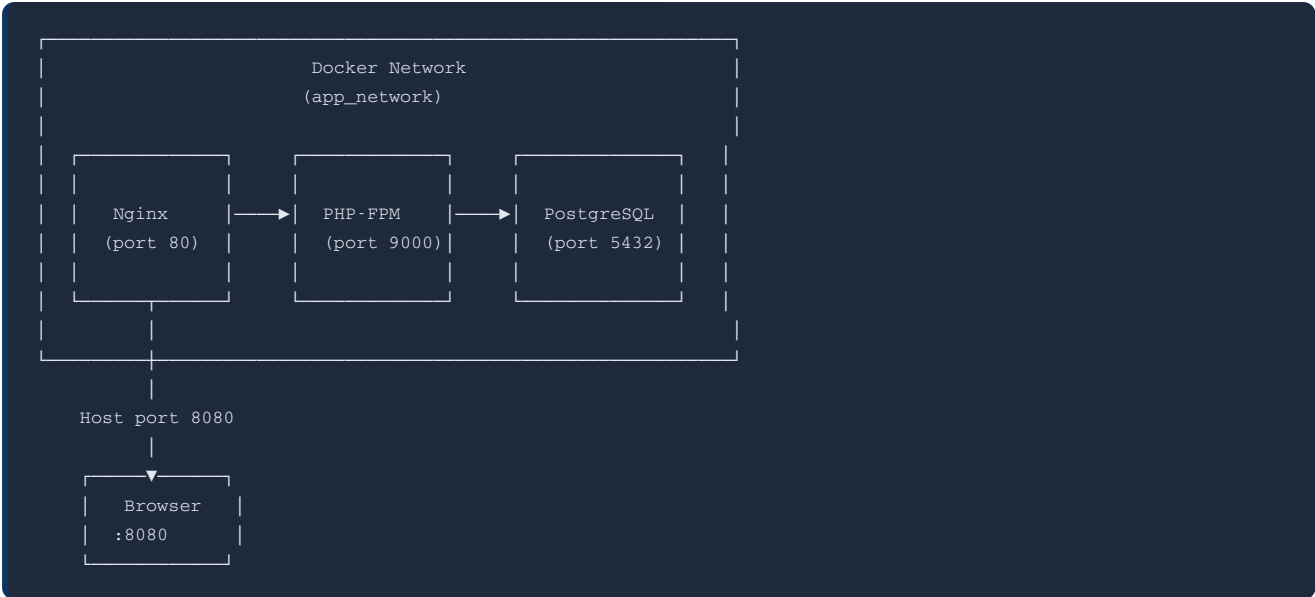
Before deploying, ensure the following tools are installed on your system:

| Tool           | Minimum Version | Check Command                       |
|----------------|-----------------|-------------------------------------|
| Docker         | 24.0+           | <code>docker --version</code>       |
| Docker Compose | 2.20+ (plugin)  | <code>docker compose version</code> |
| Git            | 2.30+           | <code>git --version</code>          |

**Note:** Docker Compose V2 is used (as a Docker plugin). If you have the standalone `docker-compose` binary, replace `docker compose` with `docker-compose` in all commands.

### Architecture Overview

The application runs as three Docker containers working together:



### Container Descriptions

| Container                        | Image                           | Purpose                            |
|----------------------------------|---------------------------------|------------------------------------|
| <code>invitation_app</code>      | Custom (PHP 8.3)                | Laravel application with PHP-FPM   |
| <code>invitation_nginx</code>    | <code>nginx:1.27-alpine</code>  | Reverse proxy, serves static files |
| <code>invitation_postgres</code> | <code>postgres:16-alpine</code> | PostgreSQL 16 database             |

### Key Features

- **Multi-stage Docker build** — Optimized image size (~200MB vs ~500MB)
- **Automatic migrations** — Database migrations run on container start
- **Health checks** — PostgreSQL readiness check before app starts
- **Security hardened** — Nginx security headers, no exposed DB port
- **Persistent data** — Database and storage data survive container restarts

## Quick Start

### Step 1: Clone the Repository

```
git clone <repository-url> invitation-management-application
cd invitation-management-application
```

### Step 2: Initialize Git Submodules

The application uses git submodules for its modules. Initialize them:

```
git submodule update --init --recursive
```

### Step 3: Configure Environment

Copy the Docker environment template and customize it:

```
cp .env.docker .env
```

**Edit** `.env` and update the following values:

```
# IMPORTANT: Change the database password to something secure!
DB_PASSWORD=your_strong_password_here

# Set your application URL (change for production)
APP_URL=http://localhost:8080
```

## Step 4: Build and Start

```
docker compose up -d --build
```

This will: 1. Build PHP dependencies (Composer) 2. Build frontend assets (Vite + Tailwind CSS) 3. Create the production PHP-FPM image 4. Start PostgreSQL with health checks 5. Run database migrations automatically 6. Optimize Laravel caches 7. Start Nginx reverse proxy

## Step 5: Create Admin User

Once all containers are running:

```
docker compose exec app php artisan shield:super-admin
```

## Step 6: Access the Application

Open your browser and navigate to:

```
http://localhost:8080
```

The admin panel is available at:

```
http://localhost:8080/admin
```

---

## Configuration

### Environment Variables Reference

| Variable                      | Default                 | Description                                                  |
|-------------------------------|-------------------------|--------------------------------------------------------------|
| <code>APP_NAME</code>         | BAGI INVITATION         | Application display name                                     |
| <code>APP_ENV</code>          | production              | Environment ( <code>local</code> , <code>production</code> ) |
| <code>APP_KEY</code>          | <i>(auto-generated)</i> | Encryption key (generated on first start)                    |
| <code>APP_DEBUG</code>        | false                   | Debug mode (set <code>false</code> for production)           |
| <code>APP_URL</code>          | http://localhost:8080   | Public URL of the application                                |
| <code>DB_HOST</code>          | postgres                | Database host (Docker service name)                          |
| <code>DB_PORT</code>          | 5432                    | Database port                                                |
| <code>DB_DATABASE</code>      | bagi_invitation         | Database name                                                |
| <code>DB_USERNAME</code>      | postgres                | Database user                                                |
| <code>DB_PASSWORD</code>      | <i>(required)</i>       | Database password                                            |
| <code>LOG_LEVEL</code>        | error                   | Logging level                                                |
| <code>CACHE_STORE</code>      | database                | Cache driver                                                 |
| <code>SESSION_DRIVER</code>   | database                | Session driver                                               |
| <code>QUEUE_CONNECTION</code> | database                | Queue driver                                                 |

## Docker Compose Port Mapping

| Service    | Container Port | Host Port     | Configurable                         |
|------------|----------------|---------------|--------------------------------------|
| Nginx      | 80             | 8080          | Edit <code>docker-compose.yml</code> |
| PostgreSQL | 5432           | <i>(none)</i> | Internal only                        |

**Security Note:** PostgreSQL is intentionally **not** exposed to the host. It is only accessible from within the Docker network. If you need direct DB access for debugging, temporarily add a port mapping in `docker-compose.yml` .

## Building & Running

### Build the Image

```
# Build with progress output
docker compose build

# Build with no cache (clean rebuild)
docker compose build --no-cache
```

### Start Services

```
# Start in detached mode (background)
docker compose up -d

# Start with build
docker compose up -d --build

# Start with logs visible
docker compose up
```

## Stop Services

```
# Stop all containers
docker compose down

# Stop and remove volumes (WARNING: deletes database data!)
docker compose down -v
```

## View Logs

```
# All services
docker compose logs -f

# Specific service
docker compose logs -f app
docker compose logs -f nginx
docker compose logs -f postgres
```

## Check Service Status

```
docker compose ps
```

Expected output when running correctly:

| NAME                | STATUS       | PORTS                |
|---------------------|--------------|----------------------|
| invitation_app      | Up (healthy) |                      |
| invitation_nginx    | Up           | 0.0.0.0:8080->80/tcp |
| invitation_postgres | Up (healthy) | 5432/tcp             |

---

## Common Operations

---

### Running Artisan Commands

```
# Run any artisan command inside the app container
docker compose exec app php artisan <command>

# Examples:
docker compose exec app php artisan migrate:status
docker compose exec app php artisan db:seed
docker compose exec app php artisan tinker
docker compose exec app php artisan cache:clear
docker compose exec app php artisan shield:super-admin
```

### Database Operations

```

# Check migration status
docker compose exec app php artisan migrate:status

# Run new migrations
docker compose exec app php artisan migrate --force

# Rollback last migration
docker compose exec app php artisan migrate:rollback

# Fresh migration (WARNING: drops all tables!)
docker compose exec app php artisan migrate:fresh --force

# Access PostgreSQL CLI
docker compose exec postgres psql -U postgres -d bagi_invitation

```

## Database Backup & Restore

```

# Backup database to a file
docker compose exec postgres pg_dump -U postgres bagi_invitation > backup_$(date +%Y%m%d_%H%M%S).sql

# Restore database from a file
docker compose exec -T postgres psql -U postgres -d bagi_invitation < backup_20260615_120000.sql

```

## Queue Worker

If you need to process queued jobs, run a worker inside the app container:

```
docker compose exec app php artisan queue:work --queue=default --tries=3 --timeout=300
```

For persistent queue processing, you can add a dedicated worker service to `docker-compose.yml`:

```

worker:
  build:
    context: .
    dockerfile: docker/php/Dockerfile
  container_name: invitation_worker
  restart: unless-stopped
  working_dir: /var/www/html
  env_file:
    - .env
  volumes:
    - storage_data:/var/www/html/storage
  networks:
    - app_network
  depends_on:
    postgres:
      condition: service_healthy
  entrypoint: ["php", "artisan", "queue:work", "--queue=default", "--tries=3", "--timeout=300"]

```

## Clearing Caches

```

docker compose exec app php artisan config:clear
docker compose exec app php artisan route:clear
docker compose exec app php artisan view:clear
docker compose exec app php artisan cache:clear

```

## Rebuilding After Code Changes

```
# Rebuild and restart
docker compose up -d --build

# Rebuild only the app image
docker compose build app
docker compose up -d
```

---

## Troubleshooting

---

### Build Fails: “composer.lock out of sync”

#### Error:

```
The lock file is not up to date with the latest changes in composer.json.
```

**Solution:** Update the lock file on your host machine:

```
composer update --lock
# Then rebuild
docker compose up -d --build
```

### Build Fails: “dubious ownership in repository”

#### Error:

```
fatal: detected dubious ownership in repository at '/var/www/html'
```

**Solution:** This is handled automatically in the Dockerfile. If it persists, ensure you’re using the latest Dockerfile with:

```
RUN git config --global --add safe.directory /app
```

### Container Keeps Restarting

Check the logs:

```
docker compose logs app
```

Common causes: - Missing `.env` file → Copy `.env.docker` to `.env` - Wrong database credentials → Check `DB_PASSWORD` in `.env` - PostgreSQL not ready → The healthcheck should handle this, but check PostgreSQL logs: `bash docker compose logs postgres`

### Cannot Access Application (Connection Refused)

1. Check if containers are running: `docker compose ps`
2. Check if port 8080 is available: `lsof -i :8080`
3. Check Nginx logs: `docker compose logs nginx`
4. Verify Nginx config: `docker compose exec nginx nginx -t`

### Permission Errors on Storage

```
docker compose exec app chown -R www-data:www-data storage bootstrap/cache
docker compose exec app chmod -R 775 storage bootstrap/cache
```

## Database Connection Refused

```
# Check PostgreSQL health
docker compose exec postgres pg_isready -U postgres

# Check environment variables
docker compose exec app env | grep DB_
```

## Static Assets Not Loading (404)

Frontend assets are built during the Docker build process. If they're missing:

```
# Rebuild the image to recompile assets
docker compose build --no-cache app
docker compose up -d
```

---

## Production Deployment

### Security Checklist

Before deploying to production, ensure:

- ☐ **Strong database password** — Change `DB_PASSWORD` from the default
- ☐ **APP\_DEBUG=false** — Never enable debug in production
- ☐ **APP\_ENV=production** — Set environment to production
- ☐ **APP\_KEY generated** — Runs automatically on first start
- ☐ **HTTPS configured** — Use a reverse proxy (e.g., Traefik, Caddy) or SSL cert
- ☐ **LOG\_LEVEL=error** — Reduce log verbosity for production
- ☐ **Firewall rules** — Only expose port 80/443 to the public
- ☐ **Regular backups** — Schedule database backups

### SSL/TLS with Reverse Proxy

For production, place an SSL-terminating reverse proxy in front of Nginx. Example with **Traefik** — add labels to the nginx service:

```
nginx:
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.invitation.rule=Host(`your-domain.com`)"
    - "traefik.http.routers.invitation.entrypoints=websecure"
    - "traefik.http.routers.invitation.tls.certresolver=letsencrypt"
```

Or use **Caddy** as a reverse proxy:

```
your-domain.com {
  reverse_proxy localhost:8080
}
```

### Scaling

To scale the PHP-FPM workers:

```
# Run 3 instances of the app service
docker compose up -d --scale app=3
```



**Note:** When scaling, you'll need to update the Nginx configuration to load balance between instances, or use Docker's built-in DNS round-robin.

---

## Maintenance

---

### Updating the Application

```
# Pull latest code
git pull origin main
git submodule update --init --recursive

# Rebuild and restart
docker compose up -d --build

# Migrations run automatically via entrypoint
```

### Viewing Container Resource Usage

```
docker stats
```

### Pruning Old Images

```
# Remove unused images
docker image prune -f

# Remove all stopped containers, unused networks, and dangling images
docker system prune -f
```

### Log Rotation

Docker logs can grow large. Configure log rotation in `/etc/docker/daemon.json`:

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

Then restart Docker:

```
sudo systemctl restart docker
```

---

## File Structure Reference

---

```
invitation-management-application/
├─ .dockerignore           # Files excluded from Docker build context
├─ .env                    # Environment configuration (create from .env.docker)
├─ .env.docker             # Docker environment template
├─ docker-compose.yml      # Docker Compose service definitions
├─ docker/
│   └─ nginx/
│       └─ default.conf    # Nginx configuration with security headers
│       └─ php/
│           └─ Dockerfile   # Multi-stage PHP-FPM build
│           └─ entrypoint.sh # Container startup script
├─ DOCKER_DEPLOYMENT_GUIDE.md # This guide
└─ ...                     # Laravel application files
```

---

## Support

---

If you encounter issues not covered in this guide:

1. Check the [Troubleshooting](#) section
2. Review container logs: `docker compose logs -f`
3. Ensure all prerequisites are installed and up-to-date
4. Verify `.env` configuration matches your environment